

A User-Centred, From-Scratch Scientific Calculator for the Tangent Function $\tan(x)$

Deliverable 1: Persona, Requirements, Algorithms, CLI Prototype

Risat Ahmed

Student ID: 40294116

SOEN 6011 – Software Engineering Processes, Section CC
Concordia University – Summer 2026

July 15, 2026

Contents

1	Introduction	3
2	Problem 1 — Persona	3
2.1	GAI tool and prompt	3
2.2	Template-selection mind map	3
2.3	Persona card: Maya Chen	3
3	Problem 2 — Requirements (ISO/IEC/IEEE 29148)	4
3.1	GAI tool and prompt	4
3.2	Requirements baseline v0.1	4
3.3	Quality review findings	5
4	Problem 3 — Two Algorithms in Pseudocode	7
4.1	GAI tool and prompt	7
4.2	Algorithm A — range reduction + Maclaurin series	7
4.3	Algorithm B — CORDIC (circular mode, rotation mode)	8
4.4	How A and B differ	9
5	Problem 4 — Algorithm Selection + Python CLI	9
5.1	GAI tool and prompt	9
5.2	Selection mind map	9
5.3	Python CLI implementation	11
5.4	Sample runs	11
6	Verification (Step 6)	12
7	Decision Rationale	13
8	Implementation Summary	13
A	GAI Prompt Log (summary)	14

1 Introduction

This report documents Deliverable 1 (D1) of a medium-sized scientific software engineering project: a user-centred, eventually from-scratch, scientific calculator implementing the real tangent function

$$\tan(x) = \frac{\sin(x)}{\cos(x)}, \quad x \in \mathbb{R},$$

with x expressed in radians. The tangent function has period π , is odd ($\tan(-x) = -\tan(x)$), has range equal to all real numbers, and is undefined at the poles $x = \pi/2 + k\pi$ ($k \in \mathbb{Z}$), where $\cos(x) = 0$. D1 covers four problems, each informed by the previous one: a user persona (Problem 1), requirements expressed per ISO/IEC/IEEE 29148 (Problem 2), two language-independent algorithms in pseudocode (Problem 3), and an algorithm selection plus a runnable Python command-line prototype (Problem 4).

Per constraint C-03, every problem below used a public generative AI (GAI) tool with a CASTROFF-structured prompt (Task, Restrictions, Output Format, Audience); the exact prompts, output evaluation, and verification method for each problem are given in their respective subsections and summarized in Appendix A.

Working assumptions (Step 0, pending final professor confirmation — see Section 7): angle unit is radians; the supported argument range is $|x| \leq 10^4$ radians; inputs within a small tolerance of a pole are rejected rather than computed.

2 Problem 1 — Persona

2.1 GAI tool and prompt

Tool: Claude Code (Claude, Anthropic, Sonnet 5 model), accessed 2026-07-15. **Prompt type:** Generative + decision-support (persona synthesis informed by a template-selection mind map). The full CASTROFF prompt is reproduced in Appendix A.

2.2 Template-selection mind map

Three persona-template styles were compared – goal-directed [1], role-based, and lightweight/proto-persona [2] – against relevance to a computation tool, realism, conciseness, traceability to requirements, and accessibility support. Figure 1 shows the comparison; the **goal-directed template** was selected because a single-function calculator’s design is driven entirely by what the user is trying to accomplish, and the template enforces that every persona field trace to a later requirement.

2.3 Persona card: Maya Chen

Field	Value
Name	Maya Chen
Role / job title	M.A.Sc. student, Mechanical Engineering; part-time research assistant in an undergraduate optics teaching lab
Experience level	Comfortable with algebra, trigonometry, and basic scripting; not a professional software developer

Field	Value
Goals	Get a trustworthy $\tan(x)$ value quickly during a lab session without opening a full CAS; avoid a silently wrong answer from a radians/degrees mix-up
Working environment	Shared teaching lab and personal laptop; occasionally a bare lab machine over a slow connection; prefers a lightweight terminal tool over a full IDE or CAS
Tangent-specific needs	(1) Clear statement of the expected angle unit; (2) confidence that inputs near $90^\circ/\pi/2$ won't silently return a nonsensical huge number; (3) a documented precision to judge lab-report suitability; (4) a way to sanity-check a familiar special value; (5) clear, non-cryptic error messages
Pain points	Radians/degrees confusion; distrust of results near an asymptote; limited patience for raw tracebacks; needing quick answers without a full CAS

Usage scenario: Maya is checking a refraction angle from her lab manual, given in degrees. She converts it to radians herself, enters it into the calculator (which first reminds her the input must be in radians and states the valid domain), and receives a result with documented precision. Out of curiosity she also tries an angle very close to $\pi/2$ and is relieved to see a clear "near an undefined point" message instead of a meaningless huge number.

Evidence vs. synthesis vs. assumption: the tangent function's mathematical properties cited above are drawn from Step 1 research notes, not invented. Maya's specific role is a plausible, illustrative composite (not a real interviewed person) of "an engineering, physics, or surveying student or lab/research assistant," per the task description. Full details, including the persona-to-requirement traceability table (7 traced needs, exceeding the ≥ 5 gate), are in `docs/persona_card.md`.

3 Problem 2 — Requirements (ISO/IEC/IEEE 29148)

3.1 GAI tool and prompt

Tool: Claude Code (Sonnet 5). **Prompt type:** Generative + transformational (persona needs $\rightarrow$ verifiable requirements) with review. The finalized Problem 1 persona was pasted into the prompt's placeholder (full prompt in Appendix A).

3.2 Requirements baseline v0.1

Requirements use a single statement style ("The system shall ...") with IDs grouped by category: FR (functional), VAL (input validation/domain), ACC (accuracy/numerical), USE (usability), REL (reliability), ERR (error handling), DOC (documentation). Priorities follow Must/Should/Could.

ID	Statement	Rationale (persona trace)	Pri.	Verification
FR-01	Accept a real-valued angle x entered as text.	No CAS available.	Must	Test

ID	Statement	Rationale (persona trace)	Pri.	Verification
FR-02	State that x is interpreted in radians before requesting input.	Radians/degrees confusion.	Must	Demo
FR-03	Compute $\tan(x) = \sin(x)/\cos(x)$ for any x accepted under VAL-01–04.	Core purpose.	Must	Test
FR-04	Display the computed result as text.	Quick trustworthy result.	Must	Demo
FR-05	Allow a new x without restarting.	Repeated use in one session.	Should	Test
FR-06	Allow exit via an explicit command.	Basic CLI usability.	Must	Demo
VAL-01	Reject non-numeric input; re-prompt with a message.	No cryptic crash on typos.	Must	Test
VAL-02	Reject empty input; re-prompt with a message.	Same as VAL-01.	Must	Test
VAL-03	Reject $ x $ exceeding a documented max (10^4 rad).	Argument-reduction error grows with $ x $.	Must	Test
VAL-04	Detect x within a documented tolerance of a pole and treat as invalid.	Fear of silent huge numbers near 90° .	Must	Test
ACC-01	Relative error $\leq 10^{-9}$ vs. reference values (outside pole band, within range).	Documented precision for lab-report use.	Must	Test
ACC-02	Display result to a fixed, documented number of significant figures (10).	Same as ACC-01.	Should	Inspection
USE-01	State valid domain and angle unit before requesting x .	Radians/pole confusion.	Must	Demo
USE-02	Provide a textual (CLI) interface, no GUI required.	Bare lab machine.	Must	Demo
REL-01	Return control without an unhandled crash, for any input.	No cryptic crash.	Must	Test
REL-02	Run from a terminal, no IDE dependency.	Shared/bare lab machine.	Must	Demo
ERR-01	On rejection, name the specific violated condition.	Helpful, non-generic errors.	Must	Test
ERR-02	Never propagate an unhandled Python exception for expected invalid/near-pole cases.	Same as ERR-01.	Must	Test
DOC-01	Document angle unit, supported range, pole tolerance, and displayed precision.	Trust for lab-report use.	Should	Inspection

Assumptions (kept separate from requirements): angle unit is radians (no degrees mode in this baseline); the pole tolerance itself is an implementation-time constant fixed in Problem 4/DOC-01; supported range is $|x| \leq 10^4$ rad; no requirement prescribes an algorithm, library, or language; "numeric" input means anything Python's standard float parsing accepts.

3.3 Quality review findings

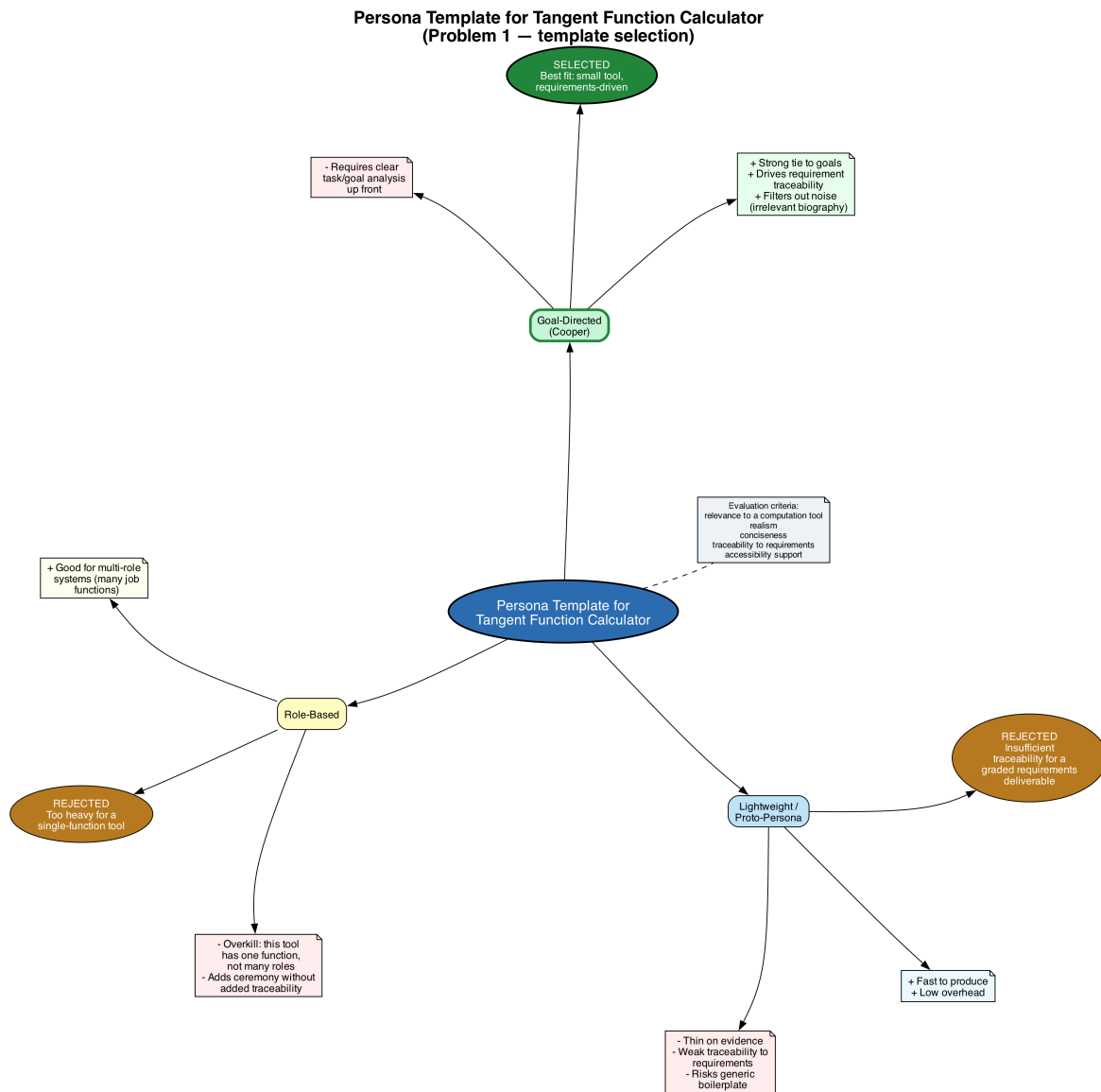


Figure 1: Persona-template selection mind map. Goal-directed template selected; role-based and lightweight/proto-persona rejected as too heavy or too thin on traceability, respectively.

ID (draft)	Characteristic at risk	Issue	Fix (adopted)
ACC-01	Verifiable / Unambiguous	"compute $\tan(x)$ quickly and accurately" – unquantified.	Quantified relative-error bound vs. a named reference.
FR-03	Feasible / no implementation prescription	Draft named "Taylor series" inside a Problem-2 requirement.	Rewritten to state only <i>what</i> is computed, not <i>how</i> .
VAL-03/04	Singular	Draft bundled range-check and pole-check in one statement.	Split into two independently verifiable requirements.

No requirement was found to violate necessary, complete, or consistent. A dedicated **PERF** (response-time) requirement was considered and deliberately deferred (Could-priority, not part of the persona's stated pain points). Frozen as **Requirements Baseline v0.1**; later changes (D2/Problem 7) will be recorded as a new version per constraint C-06.

4 Problem 3 — Two Algorithms in Pseudocode

4.1 GAI tool and prompt

Tool: Claude Code (Sonnet 5). **Prompt type:** Generative + comparative. Relevant Problem 2 requirement IDs (ACC-01, VAL-03, VAL-04, REL-01) were pasted into the prompt's placeholder (full prompt in Appendix A).

4.2 Algorithm A — range reduction + Maclaurin series

Traces to ACC-01, VAL-03, VAL-04, REL-01. Constants: π (finite-precision π), X_MAX (VAL-03), ϵ (pole tolerance, VAL-04), TOL (series tolerance), N_MAX (max terms).

Listing 1: Algorithm A: TAN-BY-SERIES

```

1 TAN-BY-SERIES(x, PI, X_MAX, EPSILON, TOL, N_MAX)
2   1 if |x| > X_MAX
3     2 signal RANGE-ERROR // VAL-03
4     3 return
5     4 s <- 1
6     5 if x < 0 // odd symmetry: tan(-x) = -tan(x)
7       6 x <- -x
8       7 s <- -s
9       8 k <- ROUND(x / (PI / 2)) // nearest multiple of pi/2
10      9 r <- x - k * (PI / 2) // r in [-pi/4, pi/4]
11     10 j <- k mod 2
12     11 sinR <- MACLAURIN-SIN(r, TOL, N_MAX)
13     12 cosR <- MACLAURIN-COS(r, TOL, N_MAX)
14     13 if j = 0
15       14 num <- sinR ; den <- cosR
16       15 else // tan(r + pi/2) = -cos(r)/sin(r)
17         16 num <- -cosR ; den <- sinR
18       17 if |den| < EPSILON
19         18 signal POLE-ERROR // VAL-04
20       19 return
21     20 return s * (num / den)
22
23 MACLAURIN-SIN(r, TOL, N_MAX)

```

```

24 1 term <- r ; sum <- r ; n <- 1
25 2 while |term| >= TOL and n <= N_MAX // max-work safeguard
26 3 term <- term * (-(r * r)) / ((2n) * (2n+1))
27 4 sum <- sum + term ; n <- n + 1
28 5 return sum
29
30 MACLAURIN-COS(r, TOL, N_MAX)
31 1 term <- 1 ; sum <- 1 ; n <- 1
32 2 while |term| >= TOL and n <= N_MAX // max-work safeguard
33 3 term <- term * (-(r * r)) / ((2n-1) * (2n))
34 4 sum <- sum + term ; n <- n + 1
35 5 return sum

```

Termination: bounded unconditionally by N_MAX regardless of tolerance. **Complexity:** $O(N_MAX)$ per call, a fixed constant (~ 10 – 15 terms for 10^{-12} relative error). **Weaknesses:** argument-reduction error grows with $|x|$ (finite-precision PI); catastrophic cancellation in the reduction subtraction for large x ; precision loss just outside the **EPSILON** guard band.

Worked trace, $\tan(\pi/4) = 1$: $k = \text{ROUND}(0.5) = 0$ (round-half-to-even), $r = \pi/4$, $j = 0$. The sine series gives $\text{sum} \approx 0.7854, 0.7046, 0.70714, 0.710270, \dots \rightarrow 0.70710678$ after a few terms; the cosine series converges symmetrically to the same value. Result: $s \cdot (\text{num}/\text{den}) = 1 \cdot (0.70710678/0.70710678) = 1$, matching the reference value.

4.3 Algorithm B — CORDIC (circular mode, rotation mode)

Same requirement trace as Algorithm A, reached by a structurally different method. Reuses the identical sign-and-quadrant reduction. Constants additionally include a precomputed table $A[i] = \arctan(2^{-i})$, computed once offline (not via a built-in `arctan` inside the algorithm).

Listing 2: Algorithm B: TAN-BY-CORDIC

```

1 TAN-BY-CORDIC(x, PI, X_MAX, EPSILON, A[0..N_MAX-1], N_MAX)
2 1 if |x| > X_MAX
3 2 signal RANGE-ERROR // VAL-03
4 3 return
5 4 s <- 1
6 5 if x < 0
7 6 x <- -x ; s <- -s
8 7 k <- ROUND(x / (PI / 2))
9 8 r <- x - k * (PI / 2) // r in [-pi/4, pi/4]
10 9 j <- k mod 2
11 10 (cosR, sinR) <- CORDIC-ROTATE(r, A, N_MAX)
12 11 if j = 0
13 12 num <- sinR ; den <- cosR
14 13 else // tan(r + pi/2) = -cos(r)/sin(r)
15 14 num <- -cosR ; den <- sinR
16 15 if |den| < EPSILON
17 16 signal POLE-ERROR // VAL-04
18 17 return
19 18 return s * (num / den)
20
21 CORDIC-ROTATE(r, A[0..N_MAX-1], N_MAX)
22 1 vx <- 1 ; vy <- 0 ; z <- r
23 2 for i <- 0 to N_MAX - 1 // fixed count: max-work safeguard
24 3 if z >= 0 : d <- +1 else d <- -1
25 4 vx_next <- vx - d * vy * 2^(-i) // shift, not divide
26 5 vy_next <- vy + d * vx * 2^(-i)

```



```

27 6 z <- z - d * A[i]
28 7 vx <- vx_next ; vy <- vy_next
29 8 K <- GAIN-CONSTANT(N_MAX) // product of cos(A[i]), precomputed
30 9 return (vx / K, vy / K) // (cos(r), sin(r))

```

Optimization note: since $\tan(r) = (K \sin r)/(K \cos r) = vy/vx$, the gain constant K cancels exactly when only $\tan$ is needed. **Termination:** exactly N_MAX iterations, unconditional. **Complexity:** $O(N_MAX)$ shift-add operations only (no multiply/divide in the loop). **Weaknesses:** linear convergence (1 bit/iteration, needing ~ 40 iterations for 10^{-12} precision, versus 10–15 series terms for Algorithm A); finite table precision caps ultimate accuracy; shares Algorithm A’s range-reduction risks.

Worked trace, $\tan(\pi/4) = 1$: starting from $(vx, vy, z) = (1, 0, 0.785398)$, the rotation sequence gives $vy/vx = 1.0, 3.0, 1.571, 1.209, 1.066, 1.0013, 0.9705, 0.9858, \dots$ after iterations 0–7, then 0.99741 at $i=9$, 0.999997 at $i=19$, and 1.000000000000 at $i=39$ – confirming both correctness and the linear-convergence weakness noted above (all values independently computed, not via a built-in $\tan$).

4.4 How A and B differ

Dimension	Algorithm A (Maclaurin)	Algorithm B (CORDIC)
Core arithmetic	Multiplication and division every term	Only addition/subtraction/shift
Convergence	Super-linear (factorial growth), ~ 10 –15 terms	Linear (~ 1 bit/iter), ~ 40 iterations
Termination	Tolerance-or-cap while loop	Fixed-count for loop
Precomputed constants	PI only	$\arctan(2^{-i})$ table (+ optional gain K)
D2 from-scratch fit	Needs from-scratch multiply/divide	Strong fit – shift/add only in hot loop
Explainability	High – "the Taylor series from calculus"	Moderate – needs a rotational mental model

The two algorithms differ in core technique, arithmetic primitives, convergence class, and termination mechanism – not superficial variants of one idea.

5 Problem 4 — Algorithm Selection + Python CLI

5.1 GAI tool and prompt

Tool: Claude Code (Sonnet 5). **Prompt type:** Decision-support (mind map) + generative (Python). Both Problem 3 algorithms were pasted into the prompt’s placeholder (full prompt in Appendix A).

5.2 Selection mind map

Selection: Algorithm A. It is the simplest way to faithfully implement an actual algorithm today, is most explainable to the persona, and has the lowest implementation effort. Algorithm B’s multiplication-free hot loop is a D2 concern, not a D1 one, so it does not outweigh A’s simplicity here; the decision will be revisited in Deliverable 2.

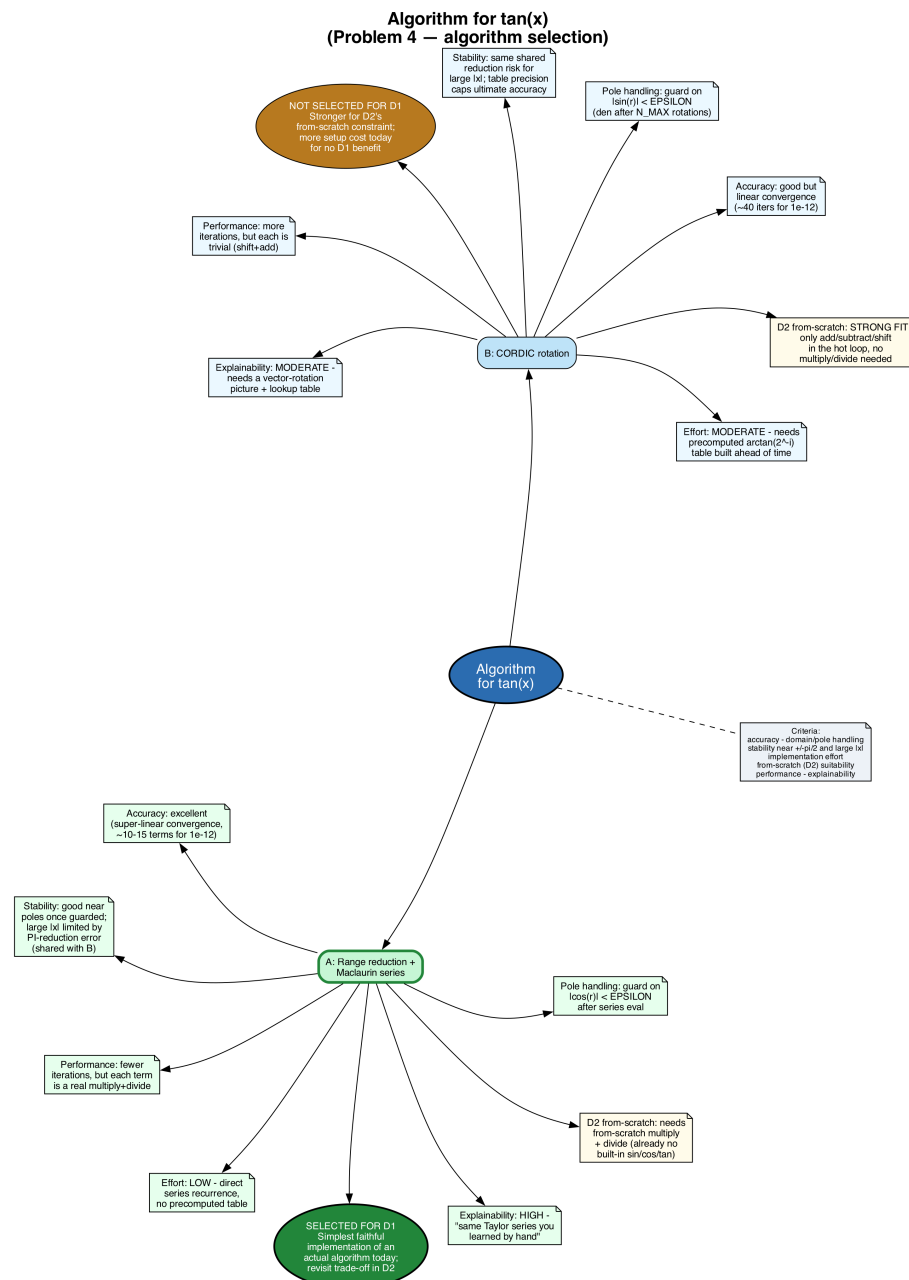


Figure 2: Algorithm-selection mind map. Algorithm A selected for the D1 prototype on implementation effort and explainability; Algorithm B noted as the stronger candidate for D2's from-scratch constraint.

5.3 Python CLI implementation

`src/cli.py` separates a pure **compute layer** (`maclaurin_sin`, `maclaurin_cos`, `tan_by_series` – implementing Algorithm A exactly) from an **I/O layer** (`run_cli`, `parse_x`, `format_result`). Two custom exceptions, `OutOfRangeError` (VAL-03) and `NearPoleError` (VAL-04), are raised by the compute layer and caught individually in the I/O loop, each producing a distinct helpful message (ERR-01) – never an unhandled traceback (ERR-02, REL-01). The angle unit and valid-domain hint are stated in the banner before the first prompt (USE-01, FR-02); results display to 10 significant figures (ACC-02, DOC-01).

Listing 3: Core compute function (excerpt), `src/cli.py`

```

1 def tan_by_series(x):
2     if abs(x) > X_MAX:
3         raise OutOfRangeError(...)
4     sign = 1.0
5     if x < 0:
6         x = -x
7         sign = -1.0 # tan(-x) = -tan(x)
8     k = round(x / (PI / 2))
9     r = x - k * (PI / 2) # r in [-pi/4, pi/4]
10    quadrant_flip = (k % 2) == 1
11    sin_r = maclaurin_sin(r)
12    cos_r = maclaurin_cos(r)
13    if quadrant_flip:
14        numerator, denominator = -cos_r, sin_r
15    else:
16        numerator, denominator = sin_r, cos_r
17    if abs(denominator) < POLE_EPSILON:
18        raise NearPoleError(...)
19    return sign * (numerator / denominator)

```

D2 from-scratch note: this implementation already computes `sin/cos` itself via the Maclaurin series; neither `math.sin` nor `math.cos` appears in the compute path. The one remaining standard-library dependency is `PI = math.pi`, which D2 must replace with an independently derived constant.

5.4 Sample runs

Case	Input	Output
Valid (special value)	$x = \pi/4$	<code>tan(0.7853981634) = 1</code>
Negative (odd symmetry)	$x = -\pi/4$	<code>tan(-0.7853981634) = -1</code>
Boundary/near-pole	$x = \pi/2 - 10^{-9}$	Cannot compute tan(x): x is at or within tolerance of a pole ...
Invalid (non-numeric)	"abc"	Invalid input: 'abc' is not a valid real number. ...
Missing input	""	Invalid input: No input received. ...
Large argument (supported)	$x = 1000$	<code>tan(1000) = 1.470324156</code>
Large argument (out of range)	$x = 10^{10}$	Input out of range: x = 1e+10 exceeds the supported range ...

Figure 3 shows two representative rendered terminal transcripts (all four, plus raw `.txt` transcripts, are in `docs/screenshots/`).

```
terminal -- python3 src/cli.py (valid + odd symmetry)

tan(x) calculator (Deliverable 1 prototype)
- Enter x as a real number, in RADIANS (not degrees).
- Supported range: |x| <= 10000 radians.
- Values at or within tolerance of a pole (x = pi/2 + k*pi) are rejected
  with an explanation rather than returning a huge/meaningless number.
- Results are displayed to 10 significant figures.
- Type 'quit' or 'exit' to leave.

Enter x (radians), or 'quit' to exit: tan(0.7853981634) = 1
Enter x (radians), or 'quit' to exit: tan(-0.7853981634) = -1
Enter x (radians), or 'quit' to exit: tan(0.5235987756) = 0.5773502692
Enter x (radians), or 'quit' to exit: tan(1.047197551) = 1.732050808
Enter x (radians), or 'quit' to exit: Goodbye.

terminal -- python3 src/cli.py (near-pole guard)

tan(x) calculator (Deliverable 1 prototype)
- Enter x as a real number, in RADIANS (not degrees).
- Supported range: |x| <= 10000 radians.
- Values at or within tolerance of a pole (x = pi/2 + k*pi) are rejected
  with an explanation rather than returning a huge/meaningless number.
- Results are displayed to 10 significant figures.
- Type 'quit' or 'exit' to leave.

Enter x (radians), or 'quit' to exit: Cannot compute tan(x): x is at or within tolerance of
  a pole (x = pi/2 + k*pi), where cos(x) is approximately zero and tan(x) is undefined.
Enter x (radians), or 'quit' to exit: Cannot compute tan(x): x is at or within tolerance of
  a pole (x = pi/2 + k*pi), where cos(x) is approximately zero and tan(x) is undefined.
Enter x (radians), or 'quit' to exit: Goodbye.
```

Figure 3: Sample CLI runs: valid values with odd symmetry (top), and near-pole rejection on both sides of a pole (bottom).

6 Verification (Step 6)

Table 6 ties requirement IDs to demo cases and results, cross-checked against `docs/research_notes.md` and an independent `math.tan` computation (never the CLI's own output used as its own reference).

Table 6: Verification matrix (abridged; full matrix in `docs/verification_matrix.md`).

Requirement ID(s)	Demo case	Reference	Verdict
FR-01,03,04,ACC-01	$x = \pi/4$	$\tan(\pi/4) = 1$	Pass

Table 6 continued

Requirement ID(s)	Demo case	Reference	Verdict
ACC-01, odd symmetry	$x = -\pi/4$	$\tan(-\pi/4) = -1$	Pass
VAL-04, ERR-01/02, REL-01	$x = \pi/2 - 10^{-9}$	undefined (pole)	Pass (rejected)
VAL-04, ERR-01/02, REL-01	$x = -\pi/2$	undefined (pole)	Pass (rejected)
VAL-01, ERR-01, REL-01	"abc"	n/a	Pass (rejected)
VAL-02, ERR-01, REL-01	" "	n/a	Pass (rejected)
ACC-01, VAL-03	$x = 1000$	1.470324156	Pass
VAL-03, ERR-01, REL-01	$x = 10^{10}$	n/a (out of range)	Pass (rejected)
FR-05/06, USE-01/02, REL-02	multi-input session + quit/EOF	clean exit both ways	Pass

Known limitations (honest disclosure): large-argument accuracy degrades gradually below the $|x| \leq 10^4$ cutoff (not yet quantified precisely, deferred to D2/D3 with a higher-precision reference); the pole tolerance (10^{-6}) and X_MAX (10^4) are provisional constants pending professor confirmation; no performance/timing requirement was verified (deliberately deferred); the reference values themselves rely on Python's finite-precision `math.tan`, adequate for this deliverable's 10^{-9} – 10^{-10} target but not an arbitrary-precision oracle.

7 Decision Rationale

Key decisions from the running decision log (`docs/decisions/decision_log.md`):

- **D-01/D-02/D-03 (provisional, pending professor confirmation):** angle unit is radians; near-pole inputs are rejected via an `EPSILON`-tolerance guard rather than computed; supported range is $|x| \leq 10^4$ rad.
- **D-04:** this deliverable's scope covers Problems 1–4 (persona, requirements, two algorithms, selection+CLI).
- **D-05:** Algorithm B (Problem 3) is CORDIC rather than the Lentz continued-fraction alternative, since CORDIC is structurally more different from Algorithm A and generalizes better to the D2 from-scratch constraint.

8 Implementation Summary

`src/cli.py` implements Algorithm A end-to-end: range reduction modulo $\pi/2$ with sign/quadrant tracking, Maclaurin series for sin/cos with a tolerance-or-cap stopping rule, a pole guard, and a textual I/O loop satisfying every Must-priority requirement in the Section 2 baseline. All sample runs in Section 4.4 and the verification matrix in Section 5 were produced by this exact source file with no modification.

A GAI Prompt Log (summary)

Per constraint C-03, the exact CASTROFF prompts (Task/Restrictions/Output Format/Audience) for all four problems are in `SOEN_6011_F2_D1_Prompts.md`, and the full evaluation log (GAI tool, output summary, critical evaluation of accepted/rejected/corrected content, verification method, attribution) is in `docs/prompts/gai_prompt_log.md`. All four problems used the same tool: Claude Code (Claude, Anthropic, Sonnet 5 model), accessed 2026-07-15. In every case, at least one part of the raw output was rejected or corrected before being adopted (documented per problem in that log) – for example, an early Problem 2 draft used unquantified terms ("fast and accurate") and was rewritten as a quantified relative-error bound, and an early Problem 4 draft called `math.sin/math.cos` directly and was corrected to call the from-scratch series functions instead, consistent with actually implementing the selected algorithm rather than wrapping a library call.

References

- [1] A. Cooper, R. Reimann, and D. Cronin, *About Face 3: The Essentials of Interaction Design*. Wiley, 2007.
- [2] J. Pruitt and T. Adlin, *The Persona Lifecycle: Keeping People in Mind Throughout Product Design*. Morgan Kaufmann, 2006.
- [3] ISO/IEC/IEEE, *ISO/IEC/IEEE 29148:2018 – Systems and Software Engineering – Life Cycle Processes – Requirements Engineering*, 2018.
- [4] J. E. Volder, "The CORDIC Trigonometric Computing Technique," *IRE Transactions on Electronic Computers*, vol. EC-8, no. 3, pp. 330–334, 1959.
- [5] J.-M. Muller et al., *Handbook of Floating-Point Arithmetic*, 2nd ed. Birkhäuser, 2016.
- [6] T. H. Cormen, C. E. Leiserson, R. L. Rivest, and C. Stein, *Introduction to Algorithms*, 3rd ed. MIT Press, 2009.
- [7] M. Abramowitz and I. A. Stegun, *Handbook of Mathematical Functions with Formulas, Graphs, and Mathematical Tables*. National Bureau of Standards, 1964.
- [8] NIST, *NIST Digital Library of Mathematical Functions*, <https://dlmf.nist.gov/>, 2026.